22/10/26

CO2 -ASSSESMENT TOOL 1

ANALYTICAL PROBLEM SOLVING

NAME: NATURAL LANGUAGE PROCESSING

COURSECODE: DSA0308

NAME: N.Umasri **REG.NO: 192424240**

1. Morphological Analysis for Document Indexing

Aim: To decompose the words connected, connecting, and connection into root and suffix components, classify the suffixes, and normalize the words for document retrieval.

Python Program

```
# Morphological Analysis for Document Indexing
words = ["connected", "connecting", "connection"]
print("Word\t\tRoot\t\tSuffix\tType\t\tNormalized")
for word in words:
    if word == "connected":
        root, suffix, kind = "connect", "ed", "Inflectional"
    elif word == "connecting":
        root, suffix, kind = "connect", "ing", "Inflectional"
    elif word == "connection":
        root, suffix, kind = "connect", "ion", "Derivational"
    print(f"{word}\t{root}\t\t{suffix}\t{kind}\t{root}")
```

Output:

```
======== RESTART: C:/Users/Hedaithulla/OneDrive/Documents/CO2 AT1 NLP.py ======
Word            Root            Suffix  Type            Normalized
connected       connect         ed      Inflectional    connect
connecting      connect         ing     Inflectional    connect
connection      connect         ion     Derivational    connect
```

Result

The three words are decomposed into the common root connect. The suffixes -ed and -ing represent grammatical forms, while -ion forms a noun through derivation. All words are normalized to connect for indexing.

2. Morphological Parsing for Sentiment Analysis

Aim: To identify the prefix, suffix, and base form of unhappy, happiness, and happily, and classify their morphological transformations.

Python Program

```
# Morphological Parsing for Sentiment Analysis

words = ["unhappy", "happiness", "happily"]

print("Word\t\tPrefix\tSuffix\tBase\tType\t\tNormalized")

for word in words:
    if word == "unhappy":
        prefix, suffix, base, kind = "un", "-", "happy", "Derivational"
    elif word == "happiness":
        prefix, suffix, base, kind = "-", "ness", "happy", "Derivational"
    elif word == "happily":
        prefix, suffix, base, kind = "-", "ily", "happy", "Derivational"
    print(f"{word}\t{prefix}\t{suffix}\t{base}\t{kind}\t{base}")
```

Output:

```
======== RESTART: C:/Users/Hedaithulla/OneDrive/Documents/CO2 AT1 NLP.py =
Word            Prefix  Suffix  Base    Type            Normalized
unhappy un      -       happy   Derivational    happy
happiness       -       ness    happy   Derivational    happy
happily -       ily     happy   Derivational    happy
```

Result

The base form of all three words is happy. The prefix un- changes the meaning to its opposite, while -ness forms a noun and -ly forms an adverb. These are derivational transformations, and the normalized root is happy.

3. Stemming-Based Preprocessing for a Search Engine

Aim: To remove suffixes from played, player, and playing, identify their stems, and distinguish grammatical inflections from word formation.

Python Program

```
# Stemming-Based Preprocessing for Search Engine

words = ["played", "player", "playing"]

print("Word\t\tStem\tAffix\tType\t\tNormalized")

for word in words:
    if word == "played":
        stem, affix, kind = "play", "ed", "Inflectional"
    elif word == "player":
        stem, affix, kind = "play", "er", "Derivational"
    elif word == "playing":
        stem, affix, kind = "play", "ing", "Inflectional"
    print(f"{word}\t{stem}\t{affix}\t{kind}\t{stem}")
```

Output:

```
======== RESTART: C:/Users/Hedaithulla/OneDrive/Documents/CO2 AT1 NLP.py =======
Word            Stem    Affix   Type            Normalized
played  play    ed      Inflectional    play
player  play    er      Derivational    play
playing play    ing     Inflectional    play
```

Result

The words played, player, and playing are reduced to the stem play. The suffixes -ed and -ing indicate grammatical forms, whereas -er forms the noun player, meaning a person who plays. All three words are normalized to play for search indexing.

4. Finite-State Morphological Parsing for Verb Inflection

Aim: To implement a finite-state parser for writes, writing, and written, identify regular and irregular inflections, and trace state transitions.

State Transition Model

State	Input or condition	Next state
q0	Start parsing	q1
q1	Recognize root write	q2
q2	Suffix s	q3
q2	Suffix ing	q4
q1	Recognize irregular form written	q5
q3, q4, q5	Complete parsing	qf

Python Program

```
# Finite-State Morphological Parser
words = ["writes", "writing", "written"]
print("Word\t\tTransition Path\t\t\tRoot\tType\t\tNormalized")
for word in words:
    if word == "writes":
        root = "write"
        kind = "Regular"
        path = "q0 -> q1 -> q2 -> q3 -> qf"
    elif word == "writing":
        root = "write"
        kind = "Regular"
        path = "q0 -> q1 -> q2 -> q4 -> qf"
    elif word == "written":
        root = "write"
        kind = "Irregular"
        path = "q0 -> q1 -> q5 -> qf"
    print(f"{word}\t{path}\t{root}\t{kind}\t{root}")
```

Output:

```
======== RESTART: C:/Users/Hedaithulla/OneDrive/Documents/CO2 AT1 NLP.py =======
Word            Transition Path                 Root    Type            Normaliz
ed
writes  q0 -> q1 -> q2 -> q3 -> qf      write   Regular write
writing q0 -> q1 -> q2 -> q4 -> qf      write   Regular write
written q0 -> q1 -> q5 -> qf    write   Irregular       write
```

Result

The finite-state parser maps all three words to the common root write. The forms writes and writing follow regular grammatical patterns, while written is an irregular past participle that requires a special rule. The transition path records how each word is recognized and normalized.

5. Porter Stemmer-Based Preprocessing for Document Retrieval

Aim: To apply Porter stemming rules to relational, relation, and relate, display intermediate forms, and obtain the final normalized stem.

Python Program

```
# Porter Stemmer-Based Preprocessing

from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = ["relational", "relation", "relate"]

print("Word\t\tStemming Rule\t\tIntermediate\tFinal Stem")

for word in words:
    if word == "relational":
        rule = "ational -> ate; remove e"
        intermediate = "relate"
    elif word == "relation":
        rule = "remove ion"
        intermediate = "relat"
    elif word == "relate":
        rule = "remove final e"
        intermediate = "relat"
    final_stem = stemmer.stem(word)
    print(f"{word}\t{rule}\t{intermediate}\t\t{final_stem}")
```

Output:

```
======== RESTART: C:/Users/Hedaithulla/OneDrive/Documents/CO2 AT1 NLP.py =======
Word            Stemming Rule           Intermediate    Final Stem
relational      ational -> ate; remove e        relate          relat
relation        remove ion      relat           relat
relate  remove final e  relat           relat
```

Result

The Porter Stemmer reduces all three words to the common stem relat. The word relational first changes to relate, and the final e is removed. The word relation loses the suffix -ion, while relate loses its final e under the applicable stemming rules. This common stem improves consistency during document indexing and retrieval.